

WGUPS Routing Program Write-Up

Matthew Sharp

008211210

msha212@wgu.edu

9 September 2024

C950 Data Structures and Algorithms II

A. Hash Table

```
HashTable.py > HashTable
1 class HashTable:
2
3     # O(1)
4     def __init__(self, size = 10): # Constructs a chaining hash table with a default size of 10
5         self._bucket_list = [[] for i in range(size)] # Creates a list of buckets based off of the provided size
6
7     # O(n)
8     def __str__(self): # String representation of the hash table
9         if self.is_empty():
10             return '{}'
11         val = '{\n'
12         for bucket in self._bucket_list:
13             if bucket:
14                 for item in bucket:
15                     val += '    ' + (str(item) + ',\n')
16
17         return val + '}'
18
19     # O(n)
20     def __iter__(self): # Allows for iteration over the hash table
21         for bucket in self._bucket_list:
22             if bucket:
23                 for item in bucket:
24                     yield item.value
25
26     # O(n)
27     def insert(self, key, value): # Insert function
28         self._get_bucket(key).append(self.Item(key, value))
29
```

B. Look-Up Function

```
31     def lookup(self, key): # Lookup function
32         bucket = self._get_bucket(key)
33         value = None
34         for item in bucket:
35             if item.key == key:
36                 value = item.value
37                 break
38         return value
39
```

C. Original Code

```
19 '''
20     Loads packages onto a specified truck.
21
22     Appends a list of packages IDs and appends them to the specified truck.
23     Accepts a string an optional that represents the time to see advance the time of the specified truck
24     and checks if the action can be performed based on the status of the specified truck.
25
26     O(N) time complexity.
27 '''
28 def load_packages(truck, package_list, time=None):
29     if time == None: # If no time is provided, the current time is used
30         time = truck.lookup('return_time')
31     else:
32         time = get_time(time)
33
34     if time < truck.lookup('return_time'): # If the time is before the truck is scheduled to return, the function is aborted
35         truck.lookup('log_event')(f'# Truck is currently on route and cannot be loaded.', time)
36         return
37
38     truck.replace('return_time', time)
39
40     for item in package_list:
41         if len(truck.lookup('packages')) >= max_packages: # Checking if the truck is full
42             truck.lookup('log_event')(f'# Truck is full, no more packages can be loaded.', time)
43             return
44
45         package = packages.lookup(item)
46
47         if package == None: # Checking if the package exists
48             truck.lookup('log_event')(f'# Package {item} does not exist.', time)
49             continue
50
51         if package.lookup('loaded_time'): # Checking if package is already loaded onto a truck
52             truck.lookup('log_event')(f'# Package {item} is already loaded onto Truck #{package.lookup("truck").lookup("id")} at {format_time(package.lookup("loaded_time"))}.'
53             package.lookup('log_event')(f'# Cannot be loaded onto Truck #{truck.lookup("id")} as it is already loaded on to Truck #{package.lookup("truck").lookup("id")} since
54             continue
55
56         truck.lookup('packages').append(package)
57         package.replace('loaded_time', True)
58         truck.lookup('log_event')(f'- Package {item} loaded.', time)
59         package.lookup('log_event')(f'- Loaded onto Truck #{truck.lookup("id")}.', time)
60         package.replace('loaded_time', time)
61         package.replace('truck', truck)
```

```

89 def generate_route(truck, time=None):
117     destination.insert('address', addresses.lookup(0))
118     destination.insert('package', None)
119     destination.insert('arrival_time', None)
120
121     destination_index += 1
122
123     route.append(destination) # Adding the hub to the route
124
125     while len(to_route):
126         nearest_distance = float('inf')
127         nearest_address = None
128         for package in to_route:
129             distance = get_distance(current_location, package.lookup('address'))
130             if distance < nearest_distance: # Finding the nearest package address
131                 nearest_distance = distance
132                 nearest_address = package.lookup('address')
133                 next_delivery = package
134
135         current_location = nearest_address # Updating the trucks current position
136
137         destination = HashTable() # Creating a new destination hash table object
138         destination.insert('id', destination_index)
139         destination.insert('address', current_location)
140         destination.insert('package', next_delivery)
141         destination.insert('arrival_time', None)
142
143         route.append(destination)
144
145         destination_index += 1
146
147         to_route.remove(next_delivery.lookup('id')) # Removing the package from the list of packages to be delivered
148
149     destination = HashTable()
150     destination.insert('id', destination_index)
151     destination.insert('address', addresses.lookup(0))
152     destination.insert('package', None)
153     destination.insert('arrival_time', None)
154     route.append(destination) # Add return to hub to the route
155
156     truck.replace('packages', [])
157     truck.lookup('log_event')(f'- Route generated.', time)
158
159     """
160     Sends a specified truck out on it's route provided in it's route property.
161     Advances the internal time of the truck and empties out the 'route' list.

```

```

main.py > _
164
165     0(N) time complexity
166     ...
167 def depart(truck, time=None):
168     if time == None: # If no time is provided, the current time is used
169         time = truck.lookup('return_time')
170     else:
171         time = get_time(time)
172
173     if time < truck.lookup('return_time'): # If the time is before the truck is scheduled to return, the function is aborted
174         truck.lookup('log_event')(f"# Truck is currently on route and cannot depart.", time)
175         return
176
177     truck.replace('return_time', time)
178
179     if len(truck.lookup('route')) == 0: # If the truck does not have a route generated, the function is aborted
180         truck.lookup('log_event')(f"# Route has not been generated.", time)
181         return
182
183     route = truck.lookup('route')
184     truck.lookup('log_event')(f"- Departed from hub.", time)
185
186     for destination in route: # Updating the departed time for each package
187         if destination.lookup('address').lookup('id') == 0: # Skip the hub
188             continue
189         package = destination.lookup('package')
190         package.lookup('log_event')(f"- Departed from hub.", time)
191         package.replace('departed_time', time)
192
193     new_time = time
194     for i in range(1, len(route)):
195         current_location = route[i-1].lookup('address') # Setting the current location to the previous destination
196         destination = route[i].lookup('address') # Setting the destination to the next destination
197         distance = get_distance(current_location, destination) # Getting the distance between the current location and the destination
198         new_time += datetime.timedelta(hours=distance / truck_speed) # Updating the time based on the distance and speed
199         truck.replace('milage', truck.lookup('milage') + distance) # Updating the truck's milage
200
201         package = route[i].lookup('package') # Checking if package is late
202         if package:
203             package.replace('delivered_time', new_time)
204             if package.lookup('deadline') < new_time:
205                 package.replace('late', True)
206                 package.lookup('log_event')(f"# Package delivered late.", new_time)
207             else:
208                 package.lookup('log_event')(f"- Delivered.", new_time)
209
210         truck.lookup('log_event')(f'{"#" if package.lookup("late") else "-"} Delivered package {package.lookup("id")}{" late" if package.lookup("late") else ""} to {package.lookup("address").lookup("address")}. (Milage : {truck.lookup("milage")}, {i})')
211
212     truck.replace('return_time', new_time)
213     truck.replace('route', [])
214     truck.lookup('log_event')(f"- Arrived back at hub. (Milage : {truck.lookup('milage')}, {i})", new_time)
215
216     ...
217     Returns a string based off of the time that is provided.
218

```

```

332 def init_gui():
333     # Clearing the console
334     os.system('cls' if os.name == 'nt' else 'clear')
335     print('*' * 80) # Only displays header on the first run
336     print('*' + (' ' * 78) + '*')
337     print('*' + (' ' * 78) + '*')
338     print('*' + 'Western Governors University Parcel Delivery Service'.center(78) + '*')
339     print('*' + (' ' * 78) + '*')
340     print('*' + (' ' * 78) + '*')
341     print('*' * 80)
342
343     # Program Loop
344     while True:
345         print()
346         print(f'Total milage driven: {sum([truck.lookup("milage") for truck in trucks]),.1f} miles'.center(80))
347         print()
348         print('Select and option to view the status:')
349         option = input('    1. Packages\n    2. Trucks\n* Enter anything else to exit *\n\n    Option: ').strip()
350         print()
351
352         match (option):
353             case '1': # Package Statuses
354                 package_id = None
355                 package = None
356                 time = None
357
358                 os.system('cls' if os.name == 'nt' else 'clear')
359                 print()
360                 print('Select and option to view the status:')
361                 option = input('    1. One Package\n    2. All packages\n* Enter anything else to go back *\n\n    Option: ').strip()
362                 print()
363
364                 if option == '1':
365                     valid_id = True
366                     try: # Verifying the package ID. If invalid it autoselects option 2
367                         package_id = int(input('Enter a package ID: ').strip())
368
369                         package = packages.lookup(package_id)
370
371                     if package == None:
372                         valid_id = False
373
374                     except:
375                         valid_id = False
376                         package_id = None
377

```

C1. Identification Information

```
main.py > ...
1  # Matthew Sharp 008211210
2  import os
3  import csv
4  import datetime
5  from HashTable import HashTable
6
7  num_trucks = 3
8  num_drivers = 2
9
10 max_packages = 16
11 truck_speed = 18
12
13 packages = HashTable()
14 addresses = HashTable()
15 trucks = HashTable()
16 drivers = HashTable()
17 debug_log = []
18
19 '''
20     Loads packages onto a specified truck.
21
22     Appends a list of packages IDs and appends them to the specified truck.
23     Accepts a string an optional that represents the time to see advance the time of the specified truck
24     ✨ and checks if the action can be performed based on the status of the specified truck.
25
26     O(N) time complexity.
27 '''
28 def load_packages(truck, package_list, time=None):
29     if time == None: # If no time is provided, the current time is used
30         time = truck.lookup('return_time')
```

C2. Process and Flow Comments

```
496 # Program Start
497 if __name__ == '__main__':
498
499     # Reading address data from the addresses.csv
500     with open('addresses.csv', encoding='utf-8-sig') as file:
501         reader = csv.reader(file)
502
503     # Populating address hash table with address data
504     for row in reader:
505         new_address = HashTable()
506         new_address.insert('id', int(row[0]))
507         new_address.insert('name', row[1])
508         new_address.insert('address', row[2])
509         new_address.insert('get_distance', lambda address, base=new_address: get_distance(base, address))
510
511         addresses.insert(new_address.lookup('id'), new_address)
512
513
514     # Creating a distance matrix from the distances.csv
515     with open('distances.csv', encoding='utf-8-sig') as file:
516         reader = csv.reader(file)
517         matrix = []
518         for row in reader:
519             entry = []
520             for item in row:
521                 if item == '':
522                     entry.append(-1)
523                 else:
524                     entry.append(float(item))
525             matrix.append(entry)
526
527     # Filling in the empty parts of the matrix
528     for i in range(len(matrix)):
529         for j in range(len(matrix) - 1, i, -1):
530             matrix[i][j] = matrix[j][i]
531
532     # Assigning each row of distances to its respective address
533     addresses.lookup(i).insert('distances', matrix[i])
534
535
536     # Reading package data from the packages.csv
537     with open('packages.csv', encoding='utf-8-sig') as file:
538         reader = csv.reader(file)
539
540     # Populating packages hash table with package data
541     for row in reader:
542         new_package = HashTable()
```


D. Interface

```
*****
*
*
*      Western Governors University Parcel Delivery Service      *
*
*
*****

      Total milage driven: 132.2 miles

Select and option to view the status:
    1. Packages
    2. Trucks
* Enter anything else to exit *

    Option: █
```

D1. First Status Check

Report for all packages at 09:35 AM.

Status: Delivered at 08:39 AM
ID: 1
Address:
 South Salt Lake Public Works
 195 W Oakland Ave
 Salt Lake City, UT 84115
Deadline: 10:30 AM
Weight: 21lbs
Notes:
Truck: Truck #1 at 07:00 AM

Status: At the hub
ID: 2
Address:
 Columbus Library
 2530 S 500 E
 Salt Lake City, UT 84106
Deadline: 11:59 PM
Weight: 44lbs
Notes:
Truck: Not loaded onto a truck.

Status: At the hub
ID: 3
Address:
 Salt Lake City Ottinger Hall
 233 Canyon Rd
 Salt Lake City, UT 84103
Deadline: 11:59 PM
Weight: 2lbs
Notes: Can only be on truck 2
Truck: Not loaded onto a truck.

Status: At the hub
ID: 4
Address:
 Utah DMV Administrative Office
 380 W 2880 S
 Salt Lake City, UT 84115
Deadline: 11:59 PM
Weight: 4lbs
Notes:
Truck: Not loaded onto a truck.

Status: At the hub
ID: 5
Address:
 Third District Juvenile Court
 410 S State St
 Salt Lake City, UT 84111
Deadline: 11:59 PM
Weight: 5lbs
Notes:
Truck: Not loaded onto a truck.

D2. Second Status Check

Report for all packages at 10:25 AM.

Status: Delivered at 08:39 AM
ID: 1
Address:
 South Salt Lake Public Works
 195 W Oakland Ave
 Salt Lake City, UT 84115
Deadline: 10:30 AM
Weight: 21lbs
Notes:
Truck: Truck #1 at 07:00 AM

Status: At the hub
ID: 2
Address:
 Columbus Library
 2530 S 500 E
 Salt Lake City, UT 84106
Deadline: 11:59 PM
Weight: 44lbs
Notes:
Truck: Not loaded onto a truck.

Status: At the hub
ID: 3
Address:
 Salt Lake City Ottinger Hall
 233 Canyon Rd
 Salt Lake City, UT 84103
Deadline: 11:59 PM
Weight: 2lbs
Notes: Can only be on truck 2
Truck: Not loaded onto a truck.

Status: At the hub
ID: 4
Address:
 Utah DMV Administrative Office
 380 W 2880 S
 Salt Lake City, UT 84115
Deadline: 11:59 PM
Weight: 4lbs
Notes:
Truck: Not loaded onto a truck.

Status: At the hub
ID: 5
Address:
 Third District Juvenile Court
 410 S State St
 Salt Lake City, UT 84111
Deadline: 11:59 PM
Weight: 5lbs
Notes:
Truck: Not loaded onto a truck.

D3. Third Status Check

Report for all packages at 01:12 PM.

Status: Delivered at 08:39 AM
ID: 1
Address:
 South Salt Lake Public Works
 195 W Oakland Ave
 Salt Lake City, UT 84115
Deadline: 10:30 AM
Weight: 21lbs
Notes:
Truck: Truck #1 at 07:00 AM

Status: Delivered at 10:54 AM
ID: 2
Address:
 Columbus Library
 2530 S 500 E
 Salt Lake City, UT 84106
Deadline: 11:59 PM
Weight: 44lbs
Notes:
Truck: Truck #2 at 10:30 AM

Status: Delivered at 11:43 AM
ID: 3
Address:
 Salt Lake City Ottinger Hall
 233 Canyon Rd
 Salt Lake City, UT 84103
Deadline: 11:59 PM
Weight: 2lbs
Notes: Can only be on truck 2
Truck: Truck #2 at 10:30 AM

Status: Delivered at 11:01 AM
ID: 4
Address:
 Utah DMV Administrative Office
 380 W 2880 S
 Salt Lake City, UT 84115
Deadline: 11:59 PM
Weight: 4lbs
Notes:
Truck: Truck #2 at 10:30 AM

Status: Delivered at 11:48 AM
ID: 5
Address:
 Third District Juvenile Court
 410 S State St
 Salt Lake City, UT 84111
Deadline: 11:59 PM
Weight: 5lbs
Notes:
Truck: Truck #2 at 10:30 AM

E. Screenshot of Code Execution

```
*****
*
*
*      Western Governors University Parcel Delivery Service      *
*
*
*****

      Total milage driven: 132.2 miles

Select and option to view the status:
    1. Packages
    2. Trucks
* Enter anything else to exit *

Option: █
```

F1. Strengths of the Chosen Algorithm

A greedy nearest neighbor algorithm is used for routing the delivery trucks with the packages that are loaded onto it. The algorithm uses a heuristic that iteratively selects the nearest location from the current position. Some of the main advantages of this approach are that it is easy to understand and implement. In addition, it has a relatively low time complexity of $O(n^2)$.

F2. Verification of Algorithm

The algorithm allows the trucks to deliver the packages in 132.2 miles, which is below the 140 maximum mile requirement. Some of the packages have special requirements or deadlines that had to be taken into account. To meet those requirements, trucks were manually loaded with certain packages and departed at designated times, but the routing took place independently and allowed the packages to be delivered in an adequate amount of time and mileage.

F3. Other Possible Algorithms

Other possible solutions could be an exhaustive search or branch and bound approach.

F3a. Algorithm Differences

An exhaustive search would consist of determining every possible combination of locations and selecting the one with the lowest mileage. This approach guarantees an optimal solution but has an abysmal runtime complexity of $O(n!)$ which is especially inefficient for larger datasets. In contrast, the greedy nearest neighbor algorithm uses a heuristic to select the nearest location from a current point. Although an optimal approach is not guaranteed, it has a much better runtime complexity of $O(n^2)$.

An improved version of an exhaustive, brute-force, search would be branch and bound approach. This algorithm starts by finding an initial path length and setting that as the lower bound. It will then explore other branches to seek a better solution. Every partial path (branch) that exceeds the lower bound will be pruned and the next branch will be explored. If a branch is found to have a better solution then the lower bound will be updated to that branch. This approach differs from the exhaustive search because it is able to prune branches that do promise a better solution; however, it does have a worst-case runtime complexity of $O(n!)$.

G. Different Approach

If I had to take a different approach to this application, I would create distinct Package, Truck, and Address classes. Instances would be stored in an associated hash table and accessed by the ID attribute. This would make the main.py file less verbose and would promote better separation of concerns. Furthermore, the debugging process would be less ambiguous and errors would be easier to pinpoint.

H. Verification of Data Structure

The primary data structure used to meet the requirements of this assignment was a chaining hash table. This flexible data structure stores entries in key value pairs, and certain attributes can be found by using a lookup function. Package, truck, and address information were all stored in instances of these hash tables and can be located with their associated ID. In addition, fields and methods are able to be defined at runtime and the structure can easily be changed by adding or removing fields.

H1. Other Data Structures

A linked list and array are examples of other data structures that could have been used for this assignment.

H1a. Data Structure Differences

A linked list stores data in noncontiguous memory locations. It does so by using a sequence of nodes, each of which has a key, associated data, and a pointer to the next node in the sequence. To access data with a provided key, the list is traversed starting from the head (the first node) and then onto each subsequent node until the key is found. This approach differs from the chaining hash table where keys are run through a hash function and mapped to a corresponding bucket of an internal data structure. The linked list, however, is often used as the underlying data structure within the chaining hash table to handle collisions by appending entries that map to the same bucket to a list stored within the bucket.

An array stores data elements in contiguous memory locations, and elements of that data can be accessed by using the corresponding positional index. This simple data structure could have been used to in many parts of this application, as IDs and various other attributes could map to a particular index within an array. The downside to this approach is that data stored at a particular index would have to stay at that index. Additionally, arrays have to be reallocated in order to change the amount of elements that they are able to hold. This differs from a chaining hash table where data is stored in key-value pair rather than just a positional index. Arrays, however, are often used to store the buckets that hash table use to map buckets to.

I. Sources

Lysecky, R., & Vahid, F. (2018, June). C950: Data Structures and Algorithms II. zyBooks.

<https://learn.zybooks.com/zybook/WGUC950AY20182019/>

Rai, A. (2023, April). Geeks for Geeks Traveling Salesman Problem Using Branch and Bound.

<https://www.geeksforgeeks.org/traveling-salesman-problem-using-branch-and-bound-2/>